Database Processing using SQLs

To use SQL in *Delphi* you need to use a query component. We will use the ADOQuery component which you can find on the ADO tab of the Component Palette *(Delphi 7)* and in the dbGo menu of the Tool Palette *(Delphi 2010)*. The ADOQuery component provides a dataset (similar to a table) containing records which you can access similar to working with an ADOTable component.

First we will show you how to display records in a DBGrid, using an ADOQuery component.

Viewing the contents of a database table

The Music database will be used to explain how to display data in a DBGrid using an ADOQuery component. Review the scenario, the design and the example data for the Music database in Appendix B.

The following components are needed to link to the Music database:

Please note:

- Only one ADOQuery component is needed. The table(s) whose data should be displayed, will be specified with a SQL statement.
- We therefore use the name of the database when naming the components.
- All properties of all components are the same as when we used an ADOTable component, except that the ADOQuery component does not have a Table property.

Did you notice that you can access any of the tables in a database using the same ADOQuery component? All you need to do is change the name of the table in the SQL statement.

The structure of an SQL statement

Each SQL statement begins with a command in the form of a verb (something that must be done). Examples of such commands include SELECT, DELETE, UPDATE and INSERT TO. For example, to display all the fields of all the records in the CD table, we would use the following statement:

SELECT * FROM CD

A qualifier indicates the criteria to be used in the query. Examples include the FROM and WHERE qualifiers. We can for example specify that only the records of Jazz CDs should be displayed, by using the WHERE qualifier:

SELECT * FROM CD WHERE Genre = "Jazz"

Changing the SQL property dynamically

Review the design views of the tables in the Music database provided in Appendix B. The following code will add SQL statements to the SQL property of the ADOQuery component named qryMusic:

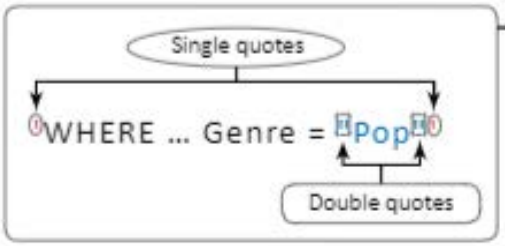
```
procedure TfrmMusic.btnEmailClick(Sender: TObject);
begin
   with dmMusic do
      begin
         qryMusic.SQL.Clear;
         qryMusic.SQL.Add('SELECT OwnerSurname, OwnerFirstName, Email FROM Owner');
         qryMusic.Open;
         pnlout.Caption := IntToStr(qryMusic.RecordCount) + ' owners.';
      end;
end;

procedure TfrmMusic.btnRockClick(Sender: TObject);
begin
   with dmMusic do
      begin
         qryMusic.SQL.Clear;
         qryMusic.SQL.Add('SELECT CDName, Artist FROM CD');
         qryMusic.SQL.Add('WHERE Genre = "Rock"');
         qryMusic.Open;
         pnlout.Caption := IntToStr(qryMusic.RecordCount) + ' Rock CDs.';
      end;
end;

procedure TfrmMusic.btnExpPopClick(Sender: TObject);
begin
   with dmMusic do
      begin
         qryMusic.SQL.Clear;
         qryMusic.SQL.Add('SELECT CDName, Artist, Genre FROM CD');
         qryMusic.SQL.Add('WHERE ReplacementValue > 150 AND Genre = "Pop"');
         qryMusic.Open;
         pnlout.Caption := IntToStr(qryMusic.RecordCount) + ' Pop CDs costing more than R150.';
      end;
end;
```

The Clear method removes all previous SQL statements from the SQL property of qryMusic.

The SQL.Add method adds SQL statements to the SQL property in the same way you entered the statements in the String List editor through the OI.

The Open method executes the SQL statements.

The Genre field is of a string data type. Therefore the value has to be included in *double* quotes. The SQL property has to contain a string, therefore the complete SQL statement is included in *single* quotes.



Notes:

- The complete SQL statement can be entered on one line, or split up by the use of additional Add methods, to improve readability.

  qryMusic.SQL.Add('SELECT OwnerSurname, ContactDetails, DiscJockey FROM Owner');

  qryMusic.SQL.Add('WHERE DiscJockey = Yes');
- SQL keywords do not have to be in capital letters, but readability is improved if they do appear in upper case. The use of capital letters is necessary in other DBMSs – so always find out what the rules are with respect to the version of SQL you are using.
- The DBGrid will display the fields for the table specified in the SQL statement.
- The Open method of the qryMusic component sets the Active property to True through the program code. In Activity 1 you set the Active property to True through the Object Inspector.
- The ADOQuery component has a RecordCount property. Every time the query is executed (through the Open method), the number of records found as a result of the query (i.e. the number of records in the resulting dataset) is stored in the RecordCount property.
- If a field name in a table contains spaces, or is the same as a reserved word in SQL (such as the name of a function), then the field name must be enclosed in square brackets, for example:

  SELECT [CD Name] FROM. . . . .

  SELECT [Date] FROM. . . . .

Formulating simple criteria

A variety of operators are available in SQL to formulate criteria. The following table contains examples of criteria which can be applied to fields in a table. Note how the syntax differs for different data types. The field names used in the examples are from the tables in the Music database.

Examples of criteria	Description and comments
The = operator	
WHERE ReplacementValue = 120	The replacement value is equal to R120.
WHERE Genre = "Rock"	The genre must be Rock. The values can be entered in lower case or upper case, but the spelling must be exactly as displayed in the data table.
WHERE DiscJockey = True WHERE DiscJockey = -1 WHERE DiscJockey = Yes	The value of the DiscJockey field is True.
WHERE DiscJockey = False WHERE DiscJockey = 0 WHERE DiscJockey = No	The value of the DiscJockey field is False.
The <> operator	
WHERE ReplacementValue <> 100	The replacement value is not equal to R100.
WHERE Genre <> "Hip Hop"	The genre is not Hip Hop.

The >, >=, < and <= operators	
WHERE ReplacementValue < 120	The replacement value is less than R120.
WHERE ReplacementValue >= 120	The replacement value is R120 or more.
WHERE OwnerSurname < "M"	The surname of the owner starts with the letters A, B,... **L**.
WHERE OwnerSurname > "M"	The surname of the owner starts with the letters **M**, N, ... Z.
The use of the = operator with > or < has no effect on text fields. Therefore, to find surnames of owners including those that start with a certain letter, for example, the letter 'M', you can use the following criteria: < "N" (Surnames that start with A, B, ... M.) > "M" (Surnames that start with M, N, ... Z.)	
The AND operator	
WHERE ReplacementValue >= 100 AND Genre = "Pop"	Pop CDs with a replacement value of R100 or more.
WHERE Grade = 11 AND Class = "B"	All owners in Grade 11B.
To find surnames of owners that start with the letter 'M' only, you can use the following criteria: WHERE OwnerSurname > "M" AND OwnerSurname < "N"	
The OR operator	
WHERE Genre = "Pop" OR Genre = "Rock"	CDs with Rock and CDs with Pop as genre.
Compound conditions with AND and OR	
WHERE Genre = "Pop" OR Genre = "Rock" AND ReplacementValue = 100	All Pop CDs, regardless of the replacement value, as well as Rock CDs with a replacement value of R100. In this case the usual precedence of logical operators applies, in that AND is evaluated before OR.
WHERE (Genre = "Pop" OR Genre = "Rock") AND ReplacementValue = 100	Pop and Rock CDs that have a replacement value of R100. The use of brackets enforces the OR to be evaluated first.
The NOT operator	
WHERE NOT Genre = "Hip Hop"	The genre is not Hip Hop. Note that this is the same as WHERE Genre <> "Hip Hop", i.e. all the genres except Hip Hop.

The IS NULL operator	
WHERE ContactDetails IS NULL	No contact details have been entered. (Null means empty.)
WHERE ContactDetails IS NOT NULL WHERE NOT ContactDetails IS NULL	Contact details have been entered.
The BETWEEN operator	
WHERE ReplacementValue BETWEEN 100 AND 120	The replacement value lies between R100 and R120, with both values included.
WHERE OwnerSurname BETWEEN "O" AND "T"	The surname of the owner starts with the letters O...**S**. (Note that the first letter is included, but the last letter is excluded.)

The LIKE operator	
The LIKE operator allows you to use wildcards in the WHERE clause of an SQL statement. This allows you to perform pattern matching. The wild cards that you can choose from are:	
%	This wild card allows you to match any string of any length (including an 'empty' string which has zero length).
_	This wild card (an underscore) allows you to match only a single character.
WHERE CDName LIKE "S%"	All CD names starting with the letter S.
WHERE CDName LIKE "%love%"	All CD names which have the word 'love' somewhere in the name.
WHERE Email LIKE "Peter__@gmail.com"	All e-mail addresses which start with the name Peter, followed by any two characters. For example, Peter11@gmail.com , Peter02@gmail.com.
The IN operator	
WHERE Genre IN ("Rock", "Pop", "Kwaito", "Jazz")	The IN operator enables you to write a query using the 'OR' operator in a shorter, more readable fashion.
The criterion using the IN operator replaces the following: WHERE Genre = "Rock" OR Genre = "Pop" OR Genre = "Kwaito" OR Genre = "Jazz"	
WHERE Grade IN (10, 11, 12)	Owners in Grades 10, 11 and 12.

In SQL, a value for the data type Date/Time is always indicated by delimiting it with the character #. (The same way a string is delimited by the character "). The date format when using the SELECT statement is always *mm/dd/yyyy* (regardless of the format set in design view in *Access*, or the format indicated in the operating system's setting—such as the Regional Settings in *Windows*). For example, the following query will display the names and birth dates of everyone with a birth date before 12 April 1990:

```
SELECT OwnerSurname, DateOfBirth
FROM Owner
WHERE DateOfBirth < #04/12/1990#
```

Leading zeroes can be omitted e.g. DateOfBirth < #4/12/1990#

The following are examples of criteria on a Date/Time field:

Examples of criteria	Description and comments
WHERE DateOfBirth = #10/02/1990#	Date of birth is 2 October 1990.
WHERE DateOfBirth < #10/02/1990#	Specify records where the date of birth is before 2 October 1990.
WHERE DateOfBirth > #4/12/1990# AND DateOfBirth < #12/31/1990#	All birth dates after 12 April 1990 and before 31 December 1990. (Both dates themselves excluded.)

WHERE DateOfBirth BETWEEN #4/12/1990# AND #12/31/1990#	All birth dates between 12 April 1990 and 31 December 1990. (Both dates themselves *in*cluded.)
WHERE [Date] LIKE "20/05/%"	• After the LIKE operator, a string should follow—even though the query uses a date. • The date format as set up in the operating system's settings should be used. (In this case it was *dd/mm/yyyy*.) • Single digit days and months should have a 0 included, e.g. 05 for the month of May.

Notes:

- Because the SQL property is a string itself, string values within the query statement need to be in quotes. For an *Access* query it doesn't matter if you use a single quote (') or a double quote ("). In *Delphi* the use of a string within a string leads to messy syntax. One single quote in a string must be indicated by two single quotes. Therefore we prefer to use double quotes to indicate string data in a SQL query in *Delphi*. Example:
  'SELECT OwnerSurname FROM Owner WHERE Grade = 12 AND Class = "A" '
  In the Music database, the Grade field is of type *number* (integer/real in *Delphi*) in the database, and the Class field is of type *text* (string/char in *Delphi*). Therefore the value 12, which is an integer, does not have to be enclosed in quotes, but the value "A", which is a string, must be enclosed in quotes.
- The DiscJockey field, in the Owner table, is of type *Yes/No* (Boolean in *Delphi*). Therefore, in an SQL statement, the value Yes does **not** have to be in quotes. It is not a string. The value Yes could also be replaced by True, or the value -1. For example:
  qryMusic.SQL.Add('SELECT OwnerSurname, ContactDetails FROM Owner');
  qryMusic.SQL.Add('WHERE DiscJockey = True');
- Field names with spaces must be enclosed in square brackets, e.g.
  SELECT [Birth Date] FROM Learners
  This field name is just an example—the databases used in this book do not have field names containing spaces. The so-called camel notation has been used, e.g. DateOfBirth.
- Remember that more complex conditions can be specified using compound conditions with the logical operators: AND, OR and NOT. The usual precedence of logical operators applies, in that AND is evaluated before OR. Be very careful therefore when mixing AND with OR, and rather use brackets to enforce the order of evaluation, as you would do in a normal If statement.

Activity 2: Use SQL statements

1. This is a written activity based on the ExcursionCape database. Write down the SQL statements for each of the following display requests. You do not need to write the *Delphi* code.
   a) Display the surnames, names, register classes and emergency contact numbers of all learners.
   b) Display the surnames, names and contact numbers of learners whose cell phone numbers start with the digits 083.
   c) Display the names, surnames and amounts paid of all learners in class A that have already made a payment.
   d) Display the names of activities for which a letter of consent is needed.
   e) Display the names, costs and dates of activities that do not cost more than R50.00.
   f) Display all details of activities which have a duration of at least two hours, but not longer than two and a half hours.
   g) Display all details of learners that want to participate in abseiling and fencing, but have not yet paid.
   h) Display all the details of activities that could be attended on the 20th of May.

2. Set up a connection to the EdwardShop database in *Delphi* and create an appropriate interface for the program. When the user clicks on a Button, a query should be executed. The result of the query should be displayed in a DBGrid. Add the Buttons and code to perform each of the following queries:
   a) **[Kg Products]**: Display all fields of all items for which the unit Kg appears in the product description.
   b) **[Cape Supplier]**: Display the contact numbers, surnames and first names of all male representatives in the Cape (*Hint*: The telephone number starts with 021).
   c) **[Baby Food]:** Display all fields of products that have the brand names Nestum, Nan or Purity, with a cost price of not more than R50.00.
   d) **[KOO]**: Display the brand name, product description and cost price of all KOO products with a cost price of no more than R60.00 and no less than R40.00.
   e) **[Bubblegum]**: Display the supplier ID, brand name and product description of all items that either have the brand name Chappies, or have the word bubblegum somewhere in the product description.

Finding unique records

The DISTINCT clause is used to obtain a list of data without any duplicates. In the ExcursionCape database, the Activity table contains duplicate dates in the Date field (more than one activity takes place on the same date). The following statements will provide a list of unique dates (so you can see a list of the dates on which activities take place):

```
with dmExcursion do
    begin
        qryExcursion.SQL.Clear;
        qryExcursion.SQL.Add('SELECT DISTINCT [Date] FROM Activity');
        qryExcursion.Open;
    end;
```

Date
20/05/2007
21/05/2007
22/05/2007
23/05/2007

You can use two fields with a DISTINCT clause. This will find all the unique *combinations* of both fields. For example, in the Learner table in the ExcursionCape database more than one learner in a class will take part in the same activity. We may want a list to see in which activities a representative of a class will participate, so that we are able to answer a query such as: 'Is there any learner in class A who will take part in abseiling?'.

```
with dmExcursion do
    begin
        qryExcursion.SQL.Clear;
        qryExcursion.SQL.Add('SELECT DISTINCT RegisterClass, ActivityName FROM Learner');
        qryExcursion.Open;
    end;
```

Through the use of the two fields in the SQL statement, it is clear that the combination of class A and abseiling does not exist, so it is clear that no learner from class A will be taking part in abseiling.

RegisterClass	ActivityName
A	Archery
A	Canopying
A	Deep sea diving
A	Fencing
A	Horseback riding
A	Kajak ride
A	River rafting
A	Snorkeling
B	Abseiling
B	Archery
B	Canopying
B	Deep sea diving
B	Fencing
B	Horseback riding
B	Kajak ride
B	River rafting
B	Snorkeling

When two fields are used with a DISTINCT clause, the combination of the two fields is reviewed. For example, many learners in class A will take part in archery, but because of the DISTINCT clause, the combination of 'A' and 'archery' is only displayed once.

Sorting records

The ORDER BY clause can be used to sort records in the DBGrid by a certain field. For example, in the ExcursionCape database we can sort records in the Learner table by the learners' surnames:

```
with dmExcursion do
  begin
    qryExcursion.SQL.Clear;
    qryExcursion.SQL.Add('SELECT Name, Surname, ActivityName FROM Learner');
    qryExcursion.SQL.Add('ORDER BY Surname');
    qryExcursion.Open;
  end;
```

However, when two learners have the same surname, their names may not be displayed in alphabetical order.

Name	Surname	ActivityName
Tanya	Maree	Abseiling
Humphrey	Mashava	Deep sea diving
Sydney	Masina	Deep sea diving
Nthabiseng	Matjila	Abseiling
Elizabeth	Matjila	River rafting
Witness	Moabela	Canopying
Boitumelo	Moabelo	Abseiling

It is possible to use more than one field in combination to sort records, for example:

```
with dmExcursion do
  begin
    qryExcursion.SQL.Clear;
    qryExcursion.SQL.Add('SELECT Name, Surname, ActivityName FROM Learner');
    qryExcursion.SQL.Add('ORDER BY Surname, Name');
    qryExcursion.Open;
  end;
```

In the previous example, Elizabeth Matjila will then be displayed before Nthabiseng Matjila.

You can specify if the records should be displayed in ascending, or descending order. For example:

```
with dmMusic do
  begin
    qryMusic.SQL.Clear;
    qryMusic.SQL.Add('SELECT OwnerSurname, Grade FROM Owner');
    qryMusic.SQL.Add('WHERE Grade IN (10, 11, 12) ORDER BY Grade DESC');
    qryMusic.Open;
  end;
```

Grade 12's will be displayed first in the DBGrid.

```
with dmMusic do
  begin
    qryMusic.SQL.Clear;
    qryMusic.SQL.Add('SELECT OwnerSurname, DateOfBirth FROM Owner');
    qryMusic.SQL.Add('WHERE DateOfBirth > #4/12/1990# ORDER BY DateOfBirth ASC');
    qryMusic.Open;
  end;
```

Smallest date (oldest learners) will be displayed first in the DBGrid.

Ascending order is the default, therefore if you do not add the qualifier ASC, it will be sorted in ascending order.

Displaying data without a DBGrid

You may want to create applications where you do not need to display data from the database in a DBGrid, but rather display summaries in another component such as a RichEdit. When a query is executed, using the SELECT statement, data is copied from the database to the dataset (such as the ADOQuery component – e.g. qryMusic).

It is possible to access each field in the dataset, in exactly the same manner as when you used an ADOTable component. For example, the following code displays output in a RichEdit:

```
procedure TfrmMusic.btnShowClick(Sender: TObject);
begin
  with dmMusic do
    begin
      qryMusic.SQL.Clear ;
      qryMusic.SQL.Add('SELECT * FROM CD') ;
      qryMusic.Open ;
      while not qryMusic.EOF do
        begin
          redOut.Lines.Add(qryMusic['Artist'] + #9 + qryMusic['CDName']);
          qryMusic.Next ;
        end;
    end;
end;
```

Determine if the current record in the dataset is the last record.

Get the next record from the dataset and make it the active record.

The output will be as follows:

Tip: Remember to set the Scrollbars property of the RichEdit to ssVertical.

DYNAMIC SQLs – input from the user

Using numbers

Display the owner name and contact details of owners in the grade as indicated by the user.

```
procedure TfrmMusic.btnDisplayGradeClick(Sender: TObject);
var
   sGrade : string ;
begin
   sGrade := InputBox('Show only owners for','Grade','') ;
   qryMusic.SQL.Clear ;
   qryMusic.SQL.Add('SELECT OwnerSurname, OwnerName, ContactDetails, Grade') ;
   qryMusic.SQL.Add('FROM Owner') ;
   qryMusic.SQL.Add('WHERE Grade = ' + sGrade) ;
   qryMusic.Open ;
end;
```

The string that should be built up for the WHERE clause will, for example, look like this:
'WHERE Grade = 12'

An Edit can be used, and we can also use code to perform data validation:

```
with dmMusic do
   begin
      try
         { Test if the number is a value between 8 and 12. }
         if not (StrToInt(edtGrade.Text) in [8..12]) then
            begin
               ShowMessage('Grade must be a number between 8 and 10 (both included).');
               edtGrade.SetFocus;
               Exit;
            end;  // if
      except  // the content of the Edit could not be converted to a number
         ShowMessage('Grade must be a valid number between 8 and 10 (both included).');
         edtGrade.SetFocus;
         Exit;
      end;  // try
      qryMusic.SQL.Clear;
      qryMusic.SQL.Add('SELECT OwnerSurname, ContactDetails FROM Owner');
      qryMusic.SQL.Add('WHERE Grade = ' + edtGrade.Text);
      qryMusic.Open;
   end;  // with
```

Please note:

Grade is a field of type integer, but the SQL statement must be a string. Therefore the variable sGrade, or the Text property of the Edit, which is a string, can just be concatenated to the rest of the SQL statement which is a string. The result will be a long string containing the grade specified by the user.

If a SpinEdit is used to indicate the grade, you will have to convert the Value property which is an integer, to a string. The code will be as follows:

```
procedure TfrmMusic.btnGradeClick(Sender: TObject);
begin
   qryMusic.SQL.Clear ;
   qryMusic.SQL.Add('SELECT OwnerName, ContactDetails') ;
   qryMusic.SQL.Add('FROM Owner') ;
   qryMusic.SQL.Add('WHERE Grade = ' + IntToStr(sedGrade.Value)) ;
   qryMusic.Open ;
end;
```

One can also use
sedGrade.Text
instead of
IntToStr(sedGrade.Value)

Using text

Obtain input from an Edit

Display the name, artist and genre of CDs, with genre as indicated by the user through the InputBox function or using an Edit

```
procedure TfrmMusic.btnDisplayGenreClick(Sender: TObject);
var
    sGenre : string ;
begin
    sGenre := InputBox('Show only CDs for','Genre','') ;
    qryMusic.SQL.Clear ;
    qryMusic.SQL.Add('SELECT Artist,CDName') ;
    qryMusic.SQL.Add('FROM CD') ;
    qryMusic.SQL.Add('WHERE Genre = "' + sGenre + '"') ;
    qryMusic.Open ;
end;
```

The string that will be built up for the WHERE clause will, for example, look like this:
'WHERE Genre = "Jazz"'

```
procedure TfrmMusic.btnGenreClick(Sender: TObject);
begin
    qryMusic.SQL.Clear ;
    qryMusic.SQL.Add('SELECT CDName, Artist, Genre ') ;
    qryMusic.SQL.Add('FROM CD') ;
    qryMusic.SQL.Add('WHERE Genre = "' + edtGenre.Text + '"') ;
    qryMusic.Open ;
end;
```

Notes:

- The string value obtained from the InputBox or from the edtGenre Edit, must be enclosed in double quotes, since the Genre field is of type string. Therefore the double quotes need to be added before and after the value obtained from the Edit.

  If the use of the quotes is confusing, you can also use the QuotedStr function, e.g:

  ```
  qryMusic.SQL.Add('WHERE Genre = ' + QuotedStr(edtGenre.Text)) ;
  qryMusic.SQL.Add('WHERE Genre = ' + QuotedStr(sGenre)) ;
  ```

  The QuotedStr function adds double quotes before and after the string.

- If the content of an Edit should be used in combination with a wild card, the wild card should be part of the string which should be enclosed in double quotes, for example:

  ```
  qryMusic.SQL.Add('WHERE Genre LIKE "% ' + edtGenre.Text + '%"') ;
  ```

  If you had to write the SQL statement, it could look as follows:

  WHERE Genre LIKE "%pop%"

Obtain input from a ComboBox

When a user has to enter values in an Edit, invalid values may be entered – for example, a user may enter the value 'Classic rock' for a genre, but none of the records in the table may have that genre. It is then better to use a ComboBox populated with actual values from a table. It is very easy to obtain a unique list of values from a table using the DISTINCT qualifier. For example, the following code will populate a ComboBox with all possible genres from the CD table in the Music database:

```
procedure TfrmMusic.FormActivate(Sender: TObject);
var
   iIndex : integer ;
begin
   with dmMusic do
      begin
         qryMusic.SQL.Add('SELECT DISTINCT Genre FROM CD') ;
         { You need to use the DISTINCT clause to ensure each genre is only entered in the ComboBox once. }
         qryMusic.Open ;
         for iIndex := 1 to qryMusic.RecordCount do
            begin
               cbxGenre.Items.Add(qryMusic['Genre']);
               qryMusic.Next ;
            end ;
         cbxGenre.ItemIndex := 0 ;
         { Display all the records in die DBGrid. }
         qryMusic.SQL.Text := 'SELECT * FROM CD' ;
         qryMusic.Open ;
      end;
end;
```

If you do not want to display anything in the DBGrid, you can set the DataSource property of the DBGrid to nil, fill the ComboBox, clear the SQL property of the query, and then set the DataSource property back to the DataSource component again.

A genre can then be selected from the ComboBox and used to build an SQL query.

```
procedure TfrmMusic.btnGenreClick(Sender: TObject);
var
    sGenre : string;
begin
    sGenre := cbxGenre.Items[cbxGenre.ItemIndex];
    with dmMusic do
        begin
            qryMusic.SQL.Clear;
            qryMusic.SQL.Add('SELECT * FROM CD WHERE Genre = ' + QuotedStr(sGenre));
            qryMusic.Open;
        end;
end;
```

Using Boolean values

Display the names and e-mail addresses of owners. Let the user indicate through use of a CheckBox whether those who are disc jockeys should be displayed or not.

```
procedure TfrmMusic.btnNamesClick(Sender: TObject);
begin
    qryMusic.SQL.Clear ;
    qryMusic.SQL.Add('SELECT OwnerName, Email') ;
    qryMusic.SQL.Add('FROM Owner') ;
    if cbxJockeys.Checked then
        qryMusic.SQL.Add('WHERE DiscJockey = Yes')
    else
        qryMusic.SQL.Add('WHERE DiscJockey = No');
    qryMusic.Open ;
end;
```

Remember that the words Yes and No can be replaced by True and False. In *Delphi* the value 0 can also be used to indicate False, and the value -1 indicates True e.g.:

```
if cbxJockeys.Checked then
    qryMusic.SQL.Add('WHERE DiscJockey = -1')
else
    qryMusic.SQL.Add('WHERE DiscJockey = 0');
```

Calculations in a query

One or more additional fields can be created, using an SQL statement, in which the result of calculations can be displayed. Note that these additional fields are only part of the table of the query – they do not become part of the underlying table in the database.

In the ExcursionCape database you can, for example, display an extra column in an SQL query to display the cost of the activity if an additional R30 per activity should be paid to include refreshments:

```
with dmExcursion do
    begin
        qryExcursion.SQL.Clear;
        qryExcursion.SQL.Add('SELECT ActivityName, Cost, Cost + 30 AS WithFood FROM Activity');
        qryExcursion.Open;
```

METHODS IN SQLs

The LCASE function	
Description	This function changes all the letters in the text data field to lower case letters.

Example	qryMusic.SQL.Clear;
	qryMusic.SQL.Add('SELECT CDName + " (" + LCASE(Genre) + ")" ');
	qryMusic.SQL.Add('AS [CD details] FROM CD');
	
	CD details
	Blue Room (rock)
	Bottles to the ground (rock)
	Break the cycle (alternative rock)
	Breakaway (pop)
	Brown (rock)
	Bury the Hatchet (pop)

The LEFT function	
Description	Returns a substring that contains a specific number of characters beginning from the left side of the data value.

Example	qryMusic.SQL.Add('SELECT OwnerSurname + " " + ');
	qryMusic.SQL.Add('LEFT(OwnerFirstname, 1) AS [Surname and initial] ');
	qryMusic.SQL.Add('FROM Owner');
	
	Surname and initial
	Ngcaba V
	Donovan P
	Curry M
	Roets K
	Braybrooke P
	Maye M

The RIGHT function	
Description	Returns a substring that contains a specific number of characters beginning from the right side of the data value.
Example	qryMusic.SQL.Add('SELECT OwnerFirstname, OwnerSurname, "Cell phone ends in "'); qryMusic.SQL.Add('+ RIGHT(ContactDetails, 4) AS CellPhone FROM Owner');

OwnerFirstname	OwnerSurname	CellPhone
Vuyolwetu	Ngcaba	Cell phone ends in 4512
Peter	Donovan	Cell phone ends in 9851
Mickey	Curry	Cell phone ends in 2008
Kevin	Roets	Cell phone ends in 8765
Piers	Braybrooke	Cell phone ends in 1123
Majorie	Maye	Cell phone ends in 5210

The LEN function	
Description	Returns the number of characters contained in a text data value.
Example	qryMusic.SQL.Add('SELECT CDName, LEN(CDName) AS CharsInTitle FROM CD');

CDName	CharsInTitle
10	2
14 shades of grey	17
A Boy named Goo	15
A New day has come	18
A Rush of Blood to the head	27
Absolution	10

The MID function	
Description	Returns a substring that contains a specific number of characters beginning from a specified character position of the data value.

Example	qryGrants.SQL.Add('SELECT LEFT(Title,1) + LCASE(MID(Title, 2, LEN(Title)-1)) +');
	qryGrants.SQL.Add('" " + PersonName + " " + Surname AS [Payment to]');
	qryGrants.SQL.Add('FROM Person');
	
	Payment to
	Ms Farah Dermody
	Mr Ellis Creswell
	Mrs Keri Handley
	Mr Ellis Derrick
	Mr Porter Mclellan
	Ms Farah Derschow
	Ms Hope Dwnn

The YEAR; MONTH; DAY functions	
Description	These functions return numeric values that represent the year of the date, the month of the year (1..12), and the day of the month (1..31).

Example	qryMusic.SQL.Add('SELECT OwnerSurname, OwnerFirstName,');
	qryMusic.SQL.Add('DAY(DateOfBirth) AS [DAY], Month(DateOfBirth) AS [MONTH],');
	qryMusic.SQL.Add('YEAR(DateOfBirth) AS [YEAR] FROM Owner');
	
	OwnerSurname, OwnerFirstName, DAY, MONTH, YEAR
	Ngcaba, Vuyolwetu, 23, 3, 1990
	Donovan, Peter, 2, 10, 1990
	Curry, Mickey, 10, 2, 1990
	Roets, Kevin, 9, 2, 1991
	Braybrooke, Piers, 12, 7, 1991
	Maye, Majorie, 6, 4, 1989

The DATE function	
Description	Determines today's date as set on the computer.

Example	qryMusic.SQL.Add('SELECT OwnerSurname, OwnerFirstName,');
	qryMusic.SQL.Add('YEAR(DATE()) - YEAR(DateOfBirth) AS Age FROM Owner');
	
	(This example determines how old someone will be at the end of the current year, not on the current day.)
	
	OwnerSurname / OwnerFirstName / Age
	Ngcaba / Vuyolwetu / 17
	Donovan / Peter / 17
	Curry / Mickey / 17
	Roets / Kevin / 16
	Braybrooke / Piers / 16
	Maye / Majorie / 18

The DATEVALUE function	
Description	Converts a string to a value in date format. The string is interpreted according to the date format setting of the operating system.

Example	qryMusic.SQL.Add('SELECT OwnerSurname, OwnerFirstName, DateOfBirth');
	qryMusic.SQL.Add('FROM Owner');
	qryMusic.SQL.Add('WHERE DateOfBirth < DateValue("5/1/1990")');
	
	The date is 5 January 1990 if the date setting is *dd/mm/yyyy*.
	
	OwnerSurname / OwnerFirstName / DateOfBirth
	Somagaca / Ayanda / 04/12/1989
	McCarthy / Liam / 15/03/1989
	Scott / Keith / 01/06/1989
	McGroggan / Henry / 13/09/1989
	Steyn / Gerhard / 15/11/1989
	Hansen / Suzanne / 24/08/1989
	
	Note the position and sequence of the day and month.
	The query produces the same result as the clause:
	WHERE DateOfBirth < #1/5/1990#

Using aggregate functions

Functions which perform calculations on fields of multiple records are called aggregate, statistical, summary or multiple-row functions.

The COUNT function	
Description	**Returns the number of records in a table or joined tables.**

Example 1
Count all records in a table.
	There are 70 owners.
	
	with dmMusic **do**
	  begin
	  qryMusic.SQL.Clear;
	  qryMusic.SQL.Add('SELECT Count(*) AS [NumOwners]');
	  qryMusic.SQL.Add('FROM Owner');
	  qryMusic.Open;
	  pnlOut.Caption := 'There are ' + IntToStr(qryMusic['NumOwners']) + ' owners.';
	  end;

Example 2
Count all records meeting certain criteria.
	There are 18 disc jockeys
	`with dmMusic do`
	`  begin`
	`  qryMusic.SQL.Clear;`
	`  qryMusic.SQL.Add('SELECT Count(*) AS [NumDJs]') ;`
	`  qryMusic.SQL.Add('FROM Owner');`
	`  qryMusic.SQL.Add('WHERE DiscJockey = True');`
	`  qryMusic.Open;`
	`  pnlOut.Caption := 'There are ' + IntToStr(qryMusic['NumDJs']) + ' disc jockeys.';`
	`  end;`

Example 3
Count non-null values in a specific field (column).
	There are 250 CDs for which replacement values have been entered.
	`with dmMusic do`
	`  begin`
	`  qryMusic.SQL.Clear;`
	`  qryMusic.SQL.Add('SELECT Count(ReplacementValue) AS [Replace]');`
	`  qryMusic.SQL.Add('FROM CD');`
	`  qryMusic.Open;`
	`  pnlOut.Caption := 'There are ' + IntToStr(qryMusic['Replace']) +`
	`  ' CDs for which replacement values have been entered.';`
	`  end;`
	
	Note: When a field name is specified as argument for the Count function, the number of non-null values in the field (column) will be returned.

The MAX and MIN functions	
Description	Returns the highest or the lowest value for a specific field. Criteria can be added for the field.

Example 1	**The most expensive CD costs R 200.00**
	with dmMusic **do**
	begin
	qryMusic.SQL.Clear;
	qryMusic.SQL.Add('SELECT MAX(ReplacementValue) AS [High]');
	qryMusic.SQL.Add('FROM CD');
	qryMusic.Open;
	pnlOut.Caption := 'The most expensive CD costs ' +
	FloatToStrF(qryMusic['High'], ffCurrency, 10, 2);
	end;

Example 2
With criteria for a field.
	The least expensive POP CD costs R 90.00
	with dmMusic **do**
	begin
	qryMusic.SQL.Clear;
	qryMusic.SQL.Add('SELECT MIN(ReplacementValue) AS [Low]');
	qryMusic.SQL.Add('FROM CD WHERE Genre = "Pop"');
	qryMusic.Open;
	pnlOut.Caption := 'The least expensive POP CD costs ' +
	FloatToStrF(qryMusic['Low'], ffCurrency, 10, 2);
	end;

The SUM function	
Description	Returns the sum of values for a specific field in various records of a table. The sum of the values for all records in a table can be calculated, or criteria can be specified so that only certain records are processed.

Example	**To replace Pearl Jam's CDs will cost R 830.00**
	with dmMusic **do**
	begin
	qryMusic.SQL.Clear;
	qryMusic.SQL.Add('SELECT SUM(ReplacementValue) AS [Total]');
	qryMusic.SQL.Add('FROM CD WHERE Artist = "Pearl Jam"');
	qryMusic.Open;
	pnlOut.Caption := 'To replace Pearl Jam''s CDs will cost ' +
	FloatToStrF(qryMusic['Total'], ffCurrency, 10, 2);
	end;

The AVERAGE function	
Description	Returns the average of values for a specific field in various records of a table. The average of the values for all records in a table can be calculated, or criteria can be specified so that only certain records are processed.

Example	**On average a CD costs R 140.04**
	with dmMusic **do**
	begin
	qryMusic.SQL.Clear;
	qryMusic.SQL.Add('SELECT AVG(ReplacementValue) AS [Average]');
	qryMusic.SQL.Add('FROM CD');
	qryMusic.Open;
	pnlOut.Caption := 'On average a CD costs ' +
	FloatToStrF(qryMusic['Average'], ffCurrency, 10, 2);
	end;

Grouping results

Performing calculations on groups of records

The GROUP BY clause can be used in conjunction with aggregate functions to return values for groups of records, instead of only one value for all records. The following are examples of *Delphi* code using SQL statements to perform calculations on groups of records.

Music database: Determine the replacement value of the CDs for each genre.

```
with dmMusic do
  begin
    qryMusic.SQL.Clear;
    qryMusic.SQL.Add('SELECT Genre, FORMAT(SUM(ReplacementValue), "Currency") AS [Total]');
    qryMusic.SQL.Add('FROM CD');
    qryMusic.SQL.Add('GROUP BY Genre');
    qryMusic.Open;
    dbgMusic.Columns[1].Width := 120;
  end;
```

Genre	Total
Afro Pop	R 300.00
Alternative Metal	R 1 750.00
Alternative Rock	R 4 610.00
Contemporary jazz	R 340.00
Hard Rock	R 380.00
Heavy metal	R 500.00

Activity 10: Group results

1. Create an application to link to the HappySnappy database. Create queries to supply the following information:

a) The number of orders processed by each assistant.

ProcessedBy	Number processed
Bongi	113
CJ	137
Clifford	151
Jackson	139
Lerato	139
Neels	150
Samson	123

b) The number of prints processed for each picture size.

PicSize	Total processed
4	12419
5	25657
J	40605
S	50667

2. Create an application to link to the ExcursionCape database. Create queries to supply the following information:

a) The number of learners who will take part in each activity.

ActivityName	Number of learners
Abseiling	9
Archery	12
Canopying	13
Deep sea diving	13
Fencing	14
Horseback riding	13
Kajak ride	13
River rafting	12
Snorkeling	12

b) The total amount of money paid by each register class per activity. Sort the results by register class.

Class activity	Total paid
A: Archery	R 135.00
A: Canopying	R 70.00
A: Deep sea diving	R 300.00
A: Fencing	R 0.00
A: Horseback riding	R 50.00
A: Kajak ride	R 45.00
A: River rafting	R 120.00
A: Snorkeling	R 45.00
B: Abseiling	R 30.00
B: Archery	R 90.00
B: Canopying	R 100.00
B: Deep sea diving	R 80.00
B: Fencing	R 135.00
B: Horseback riding	R 90.00
B: Kajak ride	R 65.00
B: River rafting	R 50.00

Applying criteria to groups of records

You can create an SQL statement to display only groups of records which meet a certain criteria. To do this you use the HAVING clause in the same way as you use a WHERE clause. Let us look at a number of examples:

Music database: Determine the replacement value of the CDs per genre for all genres where the total replacement value is more than R2000.

```
with dmMusic do
  begin
    qryMusic.SQL.Clear;
    qryMusic.SQL.Add('SELECT Genre, FORMAT(SUM(ReplacementValue), "Currency") AS [Total]');
    qryMusic.SQL.Add('FROM CD');
    qryMusic.SQL.Add('GROUP BY Genre');
    qryMusic.SQL.Add('HAVING SUM(ReplacementValue) > 2000');
    qryMusic.Open;
    dbgMusic.Columns[1].Width := 120;
  end;
```

Genre	Total
Alternative Rock	R 4 610.00
Hip Hop	R 2 140.00
Pop	R 7 145.00
Punk Pop	R 3 080.00
Rock	R 6 920.00

Grants database: Determine the total number of children per pay point. Display only the pay points which have more than 300 children.

```
with dmGrants do
  begin
    qryGrants.SQL.Clear;
    qryGrants.SQL.Add('SELECT PayPointName, SUM(Children) AS [Total Children]');
    qryGrants.SQL.Add('FROM PayPoint PP, Person PS');
    qryGrants.SQL.Add('WHERE (PP.PayPointNo = PS.PayPointNo)');
    qryGrants.SQL.Add('GROUP BY PayPointName');
    qryGrants.SQL.Add('HAVING SUM(Children) > 300');
    qryGrants.SQL.Add('ORDER BY PayPointName');
    qryGrants.Open;
  end;
```

PayPointName	Total Children
ALICEDALE	371
ALIWAL NORTH	375
BURGERSDORP	403
COFFEE BAY	451
CRADOCK	393
EAST LONDON	401
FLAGSTAFF	377
FORT BEAUFORT	381

Please note:

The aggregate function must be placed in the HAVING clause. You cannot, for example, say:

HAVING [Total children] > 300

Activity 11: Group results with criteria

Open the program you wrote in Activity 10.2. Change the queries to display the following:

a) The activities which have more than 12 learners who will participate.

b) The register class and activity where learners have not paid any money at all.

Using subqueries

Consider the Grants database. Suppose you want to display a list of names of all the people who have more than the average number of children. To do this, you would first need to determine the average number of children for all the people in the Person table. You could use a query such as the following:

```
with dmGrants do
  begin
    qryGrants.SQL.Clear;
    qryGrants.SQL.Add('SELECT FORMAT(AVG(Children),"Fixed") AS [AVERAGE]');
    qryGrants.SQL.Add('FROM Person');
    qryGrants.Open;
    dbgGrants.Columns[0].Width := 100;
  end;
```

AVERAGE
1.98

Once you know what the average is, you would then need a query such as the following:

```
SELECT PaypointName, Surname, PersonName, Children
FROM PayPoint PP, Person PS
WHERE PP.PayPointNo = PS.PayPointNo AND Children > 1.98
```

However, you cannot hard code the average value. You need to obtain the value (1.98 in this case) from another query, and then pass it on to the query displaying the list of names.

In SQL you can create a *subquery* to determine the average, and then pass the result of the subquery to a main query (the one displaying the list of names).

```
SELECT PaypointName, Surname, PersonName, Children
FROM PayPoint PP, Person PS
WHERE PP.PayPointNo = PS.PayPointNo AND Children >
(SELECT AVG(Children) FROM Person)
```

The subquery.

The main query.

The subquery (in brackets) will be executed first. The result of the subquery (in this case the value 1.98), will then be used in the main query.

In *Delphi* code, the subquery, with the query to produce the required list of names, looks as follows:

```
with dmGrants do
  begin
    qryGrants.SQL.Clear;
    qryGrants.SQL.Add('SELECT PaypointName, Surname, PersonName, Children');
    qryGrants.SQL.Add('FROM PayPoint PP, Person PS');
    qryGrants.SQL.Add('WHERE PP.PayPointNo = PS.PayPointNo');
    qryGrants.SQL.Add('AND Children >');
    qryGrants.SQL.Add('(SELECT AVG(Children) FROM Person)');
    qryGrants.Open;
  end;
```

The subquery.

PaypointName	Surname	PersonName	Children
ALICEDALE	Slaght	Zada	4
ALICEDALE	Margeson	Muhammad	4
ALICEDALE	Perciabosco	Sierra	3
ALICEDALE	Beazley	Castor	4
ALICEDALE	Caywood	Eilidh	3
ALICEDALE	Newrick	Rhoda	2
ALICEDALE	Barstad	Carlisle	4
ALICEDALE	Alexander	Brylee	2

The next query uses the HappySnappy database, and displays data of all customers who have more points than those of customer number CN011.

```
with dmHappy do
  begin
    qryHappy.SQL.Clear;
    qryHappy.SQL.Add('SELECT CustNo, CustSurname, CustName, CustPoints');
    qryHappy.SQL.Add('FROM Customer');
    qryHappy.SQL.Add('WHERE CustPoints > ');
      qryHappy.SQL.Add('(SELECT CustPoints FROM Customer');
      qryHappy.SQL.Add('WHERE CustNo = "CN011")');
    qryHappy.SQL.Add('ORDER BY CustSurname, CustName');
    qryHappy.Open;
  end;
```

The subquery.

CustNo	CustSurname	CustName	CustPoints
CN018	Adams	Orlando	3424
CN003	Ajam	Shiraz	3769
CN002	Ally	Damien	3405
CN004	Ally	Shae	3593
CN264	Appavoo	Hetal	3352
CN009	Attwell	Sarisha	3621
CN079	Behrens	Nokuthula	3035
CN306	Bili	Abigail	3244
CN309	Blaaw	Aileen	3712
CN329	Brett	Ashleigh	3469

Note: After the subquery you can still add other clauses such as the ORDER BY clause.

A subquery can also be used with groups of records and a HAVING clause. The next query uses the Grants database, and displays a list of all pay points which have more children than those in the Cradock pay point.

PaypointName	Total children
BURGERSDORP	403
COFFEE BAY	451
EAST LONDON	401
LUSIKISIKI	400
PE EAST	396
PE NORTH	421
PE WEST	395
UITENHAGE	418

```
with dmGrants do
  begin
    qryGrants.SQL.Clear;
    qryGrants.SQL.Add('SELECT PaypointName, SUM(Children) AS [Total children]');
    qryGrants.SQL.Add('FROM PayPoint PP, Person PS');
    qryGrants.SQL.Add('WHERE PP.PayPointNo = PS.PayPointNo');
    qryGrants.SQL.Add('GROUP BY PaypointName');
    qryGrants.SQL.Add('HAVING SUM(Children) > ');
      qryGrants.SQL.Add('(SELECT SUM(Children)');
      qryGrants.SQL.Add('FROM PayPoint PP, Person PS');
      qryGrants.SQL.Add('WHERE PP.PayPointNo = PS.PayPointNo');
      qryGrants.SQL.Add('AND PaypointName = "Cradock")');
    qryGrants.Open;
  end;
```

The subquery.

Note: Because you display *groups of records* which meet a certain criteria, you have to use the HAVING clause.

Activity 12: Use subqueries

Open the program which is supplied as data for this activity. The program links to the HappySnappy database.

Create the following queries to be executed when the user clicks on the respective Buttons:
(*Hint*: A subquery should be used each time.)

[Points > Avg]: Display the customer name, cell number and number of points of all customers who have accumulated more points than the average points of all the customers. Display the information sorted by the number of points in descending order.

[Totals CN010]: Customer CN010 claims to be the most loyal client and to have requested more prints in total, than any other single order. Find out if this is true. Display the customer number, as well as the total number of prints made, of all customers, where the total number of prints is more than, or equal to, the total number of prints made by customer CN010.

Using SQL to add (insert) records

Adding records to a parent table

When you added records to a parent table using an ADOTable component, you learned that:

- the primary key in the table must be a unique value
- data validation should be performed on all fields before adding a new record to a table.

The SQL statement INSERT INTO is used to add a record to a table.

The following is an example of an SQL statement that can be used to insert a record into the Owner table (the parent table) of the Music database:

```
INSERT INTO Owner
    (OwnerID, OwnerSurName, OwnerFirstName, ContactDetails, Grade, Class, Email, DiscJockey, DateOfBirth)
VALUES
    (71, "Montlantle","James", "0745531499", 8, "B", "JamesM@gmail.com", False, #5/23/1990#)
```

Notes:

- Notice the syntax for the different data types – string, number, Boolean and date.
- You do not need to add data to every field (in the example all the fields were given values).
- The data must be added in the same order as the field names.
- When you use programming code to add a new record, the values will usually be entered in controls, and the content of the controls will be used to populate the SQL property of an ADOQuery component.
- The format of the date, if you enter the actual value, is always *mm/dd/yyyy,* regardless of the operating system's settings for the date format.

Using SQL to delete records

Deleting records from a child table

The following are examples of SQL statements that can be used to delete records from the CD table in the Music database:

DELETE FROM CD	This will delete all records from the CD table.
DELETE FROM CD WHERE Artist = "Blue"	This will delete all records of songs by the artist 'Blue' from the table.
DELETE FROM CD WHERE ReplacementValue >= 200 and Genre = "Jazz"	This further refines the delete – it will delete all records of Jazz CDs where the replacement value is R200 or more.

DELETE is also an SQL statement which does not return a result set (i.e. it does not result in a table that can be displayed). Therefore the Open method, or setting the Active property to True, cannot be used. The ExecSQL method has to be called, and an additional SQL query needs to be executed to display the changed dataset.

The following code deletes all the Rock CDs from the CD table, and displays only the remaining records:

```
procedure TfrmMusic.btnDelRockClick(Sender: TObject);
begin
    qryMusic.SQL.Clear;
    qryMusic.SQL.Add( 'DELETE FROM CD WHERE Genre = "Rock"');
    qryMusic.ExecSQL;
    qryMusic.SQL.Clear;
    qryMusic.SQL.Add('SELECT * FROM CD');
    qryMusic.Open;
end;
```

Please note:

- Be careful when you use the DELETE statement. It cannot be undone and data will be lost irretrievably.
- The WHERE clause is vital, otherwise all records will be deleted from the table.
- Deleting can fail if *Access* has the table open at the same time as your *Delphi* program – so make sure that the only application that has access to the table, is your program.
- Always make a backup of the original database before you make any changes to it.

There are various ways in which you can let the user indicate which record(s) should be deleted from a table.

```
procedure TfrmMusic.btnDeleteClick(Sender: TObject);
var
   sCDName: string;
begin
   with dmMusic do
      begin
         sCDName := qryMusic['CDName']; // Obtain the CD's name from the dataset.
         { Confirm that the user wants to delete the record. }
         if MessageDlg('Are you sure you want to delete the CD: ' + sCDName + '?',
                                 mtWarning, [mbOK, mbCancel],0) = mrOK then
            begin
               qryMusic.SQL.Clear;
               qryMusic.SQL.Add('DELETE FROM CD WHERE CDName = ' + QuotedStr(sCDName));
               qryMusic.ExecSQL; // Use the ExecSQL method, not the Open method
               qryMusic.SQL.Clear;
               qryMusic.SQL.Add('SELECT Artist, CDName, Genre, ');
               qryMusic.SQL.Add('FORMAT(ReplacementValue, "Currency")');
               qryMusic.SQL.Add('AS ReplacementValue FROM CD ');
               qryMusic.Open;
               dbgMusic.Columns[3].Width := 120;
            end // if
         else
            ShowMessage(sCDName + ' will not be deleted');
      end; // with
end;
```

This shaded code is exactly the same as the code in the event handler for the OnShow event of the Form. Therefore it can be replaced by the following statement:

FormShow(Self);

The FormShow event handler is called with the argument 'Self' to indicate that the program itself called the method. This method call will cause the code in the event handler for the OnShow event of the Form to be executed.

Deleting records using a subquery

We would like to delete the CDs whose owners are in a certain grade and class (e.g. 12D). The grade and class must be obtained from the Owner table, and the records must be deleted from the CD table (so values from two different tables are used). This type of deletion requires the use of a subquery, which is part of the WHERE clause of the DELETE statement.

For example, to delete all the CDs belonging to Grade 12D, the following SQL statement can be used:

```
DELETE FROM CD
WHERE OwnerID IN (SELECT OwnerID
                  FROM Owner
                  WHERE Grade = 12 AND Class = "D")
```

Using SQL to change (update) records

The following are examples of SQL statements that can be used to update records (change the values of one or more specified fields) from the ExcursionCape database.

```
UPDATE Learner
SET AmountPaid = AmountPaid + 50
WHERE LearnerID = '8903156861321'
```

This learner paid R50, and the content of the AmountPaid field should be increased by 50.

```
UPDATE Learner
SET ActivityName = 'Archery'
WHERE ActivityName = 'Abseiling'
```

The activity Abseiling will no longer take place, and all those learners will now do Archery instead.

```
UPDATE Activity
SET [Date] = #05/26/2007#
WHERE [Date] = #05/20/2007#
```

All activities planned for 20 May 2007 have been re-scheduled, and will now take place on 26 May 2007.

```
UPDATE Activity
SET [Date] = #05/22/2007#
WHERE [Date] = ActivityName = 'Archery'
```

The date for Archery has changed to 22 May 2007. Note: There are other activities which take place on the same date as Archery (currently 23 May 2007), but their date has to stay 23 May 2007, therefore you need to specify the name of the activity, and not the current date.

Remember that the word Date is a reserved word in *Delphi* as well as in *Access*, therefore when it is used as a field name, it should be typed in square brackets.

Homework Activity 7 Integrated scenario Page 152 – Page 154